Home Assignment

Implement playwright/typescript with vibe coding

Implementation

User: analyse the code

GitHub Copilot: Reviewing the extension source and manifest to summarize architecture and behavior.

Read [](file:///c%3A/Users/JyotiAshtankar/----------/AI/Week%203/Day1/ai-extension/ai-extension/manifest.json)

Read [](file:///c%3A/Users/JyotiAshtankar/----------/AI/Week%203/Day1/ai-extension/ai-extension/bg.js)

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/panel.html)

Read [](file:///c%3A/Users/JyotiAshtankar/----------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js#1-1), lines 1 to 200

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/popup.js#1-1), lines 1 to 200

I've confirmed the extension has a side panel chat interface with inspector integration. Next I'll inspect the remaining script pieces and content script to complete the architecture summary.

Searched for regex `class ChatUI|new ChatUI|getPrompt|handleGitHubAPIRequest|SELECTED_DOM_CONTENT|chrome.scripting.executeScript|chrome.storage\.sync|providerSelect` (`**/*.{js,html}`), 35 results

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js)

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/content/content.js)

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/config/configUtils.js)

I've identified the primary flow; now I'm pulling the remaining implementation details for message handling and API wiring.

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/popup.js#200-200), lines 200 to 520

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js#200-200), lines 200 to 717

I've found the main UI logic and prompt generation flow. Now checking the API classes and any unused or duplicate content handling.

Searched for regex `class GroqAPI|class OpenAIAPI|class TestleafAPI|export class GroqAPI|export class OpenAIAPI|export class TestleafAPI` (`**/src/**/*.js`), 3 results

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/api/groq-api.js)

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/api/openai-api.js)

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/api/testleaf-api.js)

I've got the core logic and API adapters. One quick check remains: whether there is a second content script file that duplicates or conflicts with content.js.

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/content/content-script.js)

Codebase analysis

Architecture overview

- manifest.json
  - Chrome extension MV3.
  - Defines a side panel at panel.html.
  - Uses bg.js as a service worker.
  - Requests permissions: `storage`, `activeTab`, `scripting`, `tabs`, `sidePanel`.
  - Exposes some resources as web-accessible.

- bg.js
  - Opens the side panel when the extension icon is clicked.
  - Receives `SELECTED_DOM_CONTENT` messages from content script and stores the HTML in `chrome.storage.local`.
  - Handles `GITHUB_API_REQUEST` as a fallback fetch bridge.
  - Cleans up content scripts on `chrome.runtime.onSuspend`.

Side panel / UI

- panel.html

  - Tabbed UI with “Code Generator” and “Settings”.

  - Chat input area, inspector button, generate button.

  - Settings include language binding, browser engine, provider/model selection, API key fields.

  - Loads:

    - `init-config.js`

    - popup.js

    - `tabs.js`

    - groq-api.js

    - chat.js

    - openai-api.js

    - testleaf-api.js

- configUtils.js

  - Applies theme/config colors into CSS variables.

  - Called early via `init-colors.js`.

Chat and generation flow

- chat.js

  - `ChatUI` controls the chat panel.

  - Tracks inspector state, selected DOM, provider/model, generated code.

  - Listens for `SELECTED_DOM_CONTENT` messages to receive selected HTML from page inspector.

  - `sendMessage()`:

    - Validates selected DOM and provider API key.

    - Processes selected HTML, trims `<select>` options to at most 3.

    - Builds prompts using `getPrompt()` from prompts.js.

  - Sends the prompt to the selected API adapter.
  - Displays result with markdown rendering and syntax highlighting.
- Prompt selection is based on the language/browser dropdowns + feature/page checkboxes.
- Currently, only Java + Selenium is fully supported with custom prompt templates.

- prompts.js
- Defines prompt templates for:
  - `SELENIUM_JAVA_PAGE_ONLY`
  - `CUCUMBER_ONLY`
  - `CUCUMBER_WITH_SELENIUM_JAVA_STEPS`
- Contains `getPrompt()` to substitute variables like `${domContent}` and `${pageUrl}`.

Content selection / inspector

- content.js
- Inspector script injected into active tab.
- Highlights hovered elements and toggles selection on click.
- Sends selected content via `chrome.runtime.sendMessage({ type: 'SELECTED_DOM_CONTENT', content })`.
- Uses CSS classes `genai-highlight` and `genai-selected`.

- content-script.js
- A second inspector/content script with similar but not identical logic.
- Uses `.inspector-highlight` and maintains selected HTML arrays.
- Also responds to `TOGGLE_INSPECTOR`, `GET_SELECTED_DOM`, and `CLEANUP`.

API adapters

- groq-api.js
  - Calls Groq chat completions endpoint.
  - Extracts code blocks from the response.
  - Returns only `content`, no token usage metadata.

- openai-api.js
  - Calls OpenAI chat completions.
  - Returns response content and usage tokens.

- testleaf-api.js
  - Calls Testleaf chat completions.
  - Returns response content and usage tokens.

Findings / potential issues

- Duplicate inspector/content-script logic
  - Both content.js and content-script.js exist.
  - This may cause inconsistent behavior or confusion about which script is active.

- Mixed UI initialization
  - popup.js and chat.js both handle UI/state initialization.
  - popup.js includes settings and some inspector toggling, while chat.js handles chat flow.
  - There may be overlap or outdated code paths.

- Provider/model selection

- `init-config.js` only offers model lists for `groq` and `openai`.
- `claude` appears in the UI but no model options are configured for it.

- Browser API assumptions
  - bg.js uses `chrome.sidePanel.open()` which may not be available in all Chromium versions.

- Runtime behavior
  - `sendMessage()` requires DOM selection before generating code.
  - Non-Java/Selenium combos are intentionally unsupported and display a warning, but the UI still allows selecting them.

Summary

The extension is built as a side-panel code generator for test automation, with:

- page inspector DOM selection,
- prompt-based AI code generation,
- multiple LLM provider adapters,
- persistent settings storage.

The main implementation is in panel.html, chat.js, popup.js, content.js, and the API adapters.

If you want, I can next map the exact runtime call sequence or identify the best place to simplify the duplicate content script logic.

User: Implement and update the same code for playwright/typescript

Both selenium and playwright/typescript code should work

GitHub Copilot: Reviewing prompt selection and language/engine decision points now.

Searched for regex `PLAYWRIGHT|typescript|TypeScript|playwright|PROMPT|CUCUMBER_WITH_SELENIUM|SELENIUM_JAVA_PAGE_ONLY|getPromptKeys|isJavaSelenium` (`**/src/**/*.js`), 64 results

I've located the generation routing and prompt template code; next I'm adding Playwright/TypeScript prompt support and wiring it into the selection logic.

Replacing 141 lines with 269 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js)

```

```

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#180-180), lines 180 to 222

I've inserted the Playwright/TS prompt definitions; now I'm wiring the selection logic and adding the new helper function.

Replacing 52 lines with 63 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js)

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js#540-540), lines 540 to 625

Replacing 60 lines with 71 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js)

```

```

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js#360-360), lines 360 to 410

Replacing 20 lines with 25 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js)

```

```

Replacing 4 lines with 4 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js)

```

```

Replacing 12 lines with 12 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js)

```

```

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#1-1), lines 1 to 140

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#140-140), lines 140 to 222

Searched for regex `PLAYWRIGHT_TS_PAGE_ONLY|CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS` (`**/src/scripts/prompts.js`), 2 results

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#60-60), lines 60 to 130

Replacing 10 lines with 57 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js)

```

```

Replacing 5 lines with 6 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js)

```

```

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#110-110), lines 110 to 150

Searched for regex `PLAYWRIGHT_TS_PAGE_ONLY|CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS` (`**/src/scripts/prompts.js`), 3 results

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#160-160), lines 160 to 260

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#20-20), lines 20 to 75

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#75-75), lines 75 to 112

I found the Playwright page-object prompt, but the combined Playwright/Cucumber prompt still needs insertion.

Replacing 11 lines with 90 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js)

```

```

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#30-30), lines 30 to 62

Replacing 6 lines with 6 lines in [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js)

```

```

Searched for text `CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS` (`**/src/scripts/prompts.js`), 2 results

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#230-230), lines 230 to 310

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js#310-310), lines 310 to 340

Checking for syntax or lint issues in the modified prompt and chat logic.

Checked [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/prompts.js), [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js), no problems found

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js#440-440), lines 440 to 470

Read [](file:///c%3A/Users/JyotiAshtankar/---------/AI/Week%203/Day1/ai-extension/ai-extension/src/scripts/chat.js#470-470), lines 470 to 483

Done

Updated the extension to support Playwright + TypeScript alongside existing Selenium + Java flow.

What changed

- prompts.js
  - Added `PLAYWRIGHT_TS_PAGE_ONLY` prompt template.
  - Added `CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS` prompt template.
  - Kept existing `SELENIUM_JAVA_PAGE_ONLY` / `CUCUMBER_WITH_SELENIUM_JAVA_STEPS`.
  - Updated `CODE_GENERATOR_TYPES` with Playwright/TS entries.

- chat.js
  - Extended `getPromptKeys()` to return Playwright/TS prompt keys when `languageBinding === 'ts'` and `browserEngine === 'playwright'`.

- Added `isTypeScriptPlaywright()` helper.

- Added fallback handling for unsupported combos.

- Updated loader text to display `Playwright` when the Playwright engine is selected.

Result

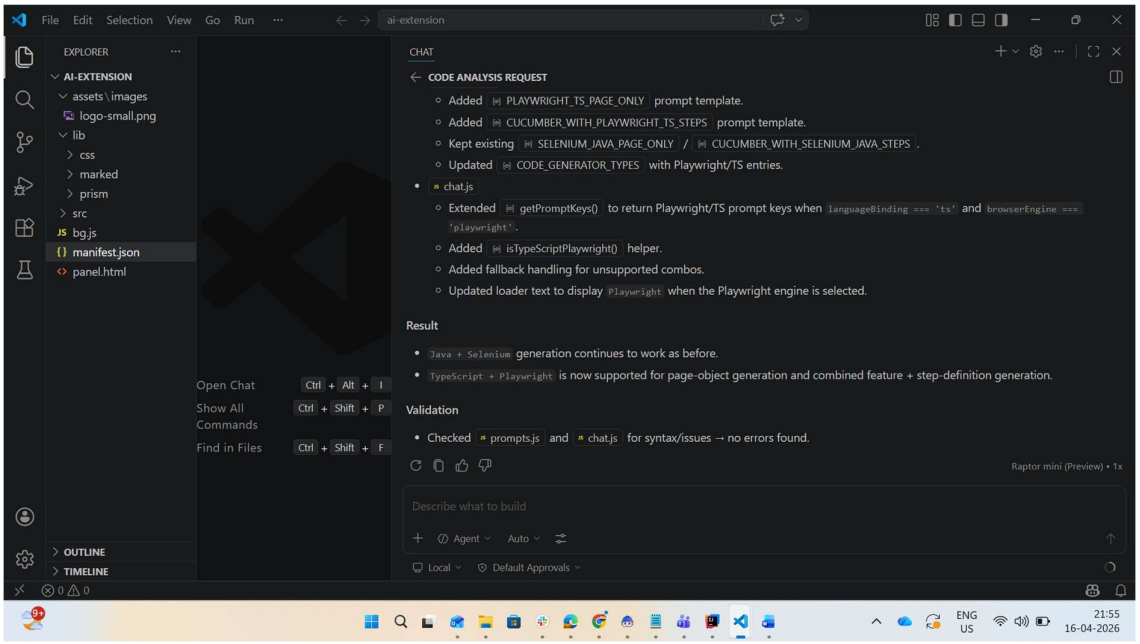
- `Java + Selenium` generation continues to work as before.

- `TypeScript + Playwright` is now supported for page-object generation and combined feature + step-definition generation.

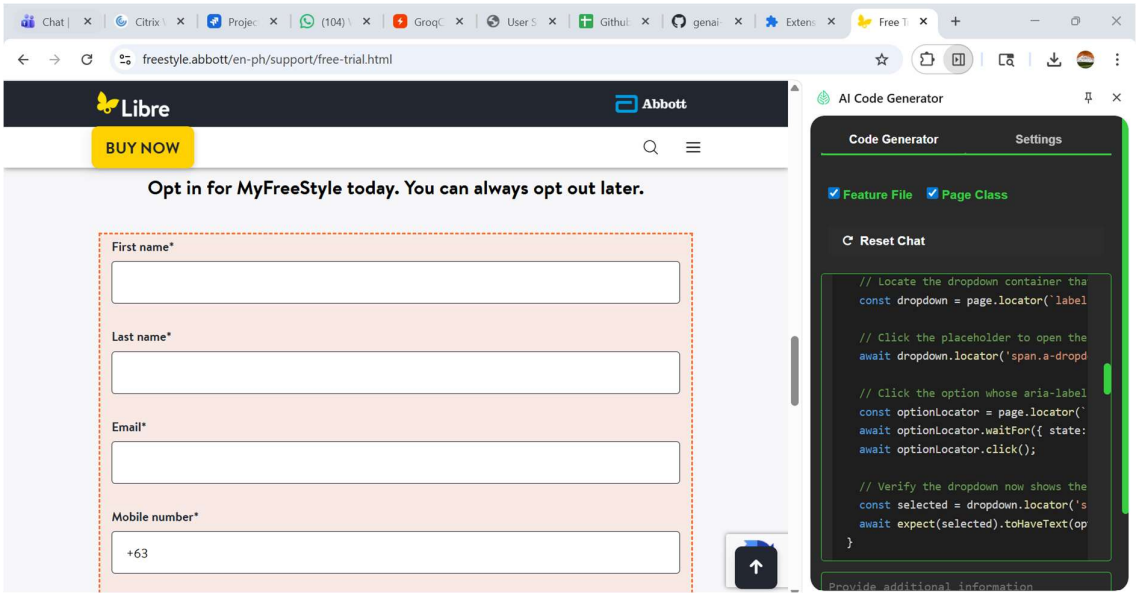
Validation

- Checked prompts.js and chat.js for syntax/issues → no errors found.

Made changes.



Implementation and use:



Generated TypeScript

Feature: FreeStyle Libre Free-Trial Form Submission

As a QA engineer I want to verify that the free-trial form can be filled

with valid data from South-Indian users and that a confirmation message

is shown after submission.

Scenario Outline: Submit free-trial form with valid data

Given I open the free-trial page

When I type "<firstName>" into the First name field

And I type "<lastName>" into the Last name field

And I type "<email>" into the Email field

And I type "<mobile>" into the Mobile number field

And I select "<treatment>" from the "What is your current diabetes treatment" dropdown

And I select "<diabetesType>" from the "What is your diabetes type" dropdown

And I select "<firstTime>" from the "Is this your first time using FreeStyle Libre?" dropdown

And I select "<purchased>" from the "Have you purchased FreeStyle Libre?" dropdown

And I select "<monitor>" from the "How do you usually monitor your Glucose?" dropdown

And I check the consent checkbox

And I click the Submit button

Then I should see a confirmation message containing "<confirmationText>"

Examples:

| firstName | lastName | email | mobile | treatment | diabetesType | firstTime | purchased | monitor | confirmationText |

| Arjun | Rao | arjun.rao@example.com | +91 9876543210 | Insulin requiring | I have Type 1 diabetes | Yes | No | Finger Prick (Glucometer) | "Thank you for your request" |

| Priya | Menon | priya.menon@example.in | +91 9123456789 | Non-insulin requiring| I have Type 2 diabetes | No | Yes | CGM | "Your submission has been received" |

| Sandeep | Kumar | sandeep.kumar@example.co.in| +91 9988776655 | Insulin requiring | I am a parent of someone managing diabetes | Yes | No | Lab Testing (Fasting) | "Thank you for your request" |

```
import { Given, When, Then } from '@cucumber/cucumber';

import { chromium, Browser, Page } from 'playwright';

import { expect } from '@playwright/test';


let browser: Browser;

let page: Page;


/**

 * Helper to select an option from a dropdown identified by its label text.
```

```
 */
async function selectFromDropdown(label: string, option: string) {
  // Locate the dropdown container that has the given label
  const dropdown = page.locator(` label:has-text("${label}") `).locator('..').first();

  // Click the placeholder to open the list
  await dropdown.locator('span.a-dropdown__placeholder').click();

  // Click the option whose aria-label matches the desired text (trimmed)
  const optionLocator = page.locator(` li[aria-label="${option.trim()}"] `);
  await optionLocator.waitFor({ state: 'visible', timeout: 5000 });
  await optionLocator.click();

  // Verify the dropdown now shows the selected value
  const selected = dropdown.locator('span.a-dropdown__placeholder');
  await expect(selected).toHaveText(option.trim());
}

/**
 * Open the Free-Trial page.
 */
Given('I open the free-trial page', async () => {
  browser = await chromium.launch({ headless: true });
  page = await browser.newPage();
  await page.goto('https://www.freestyle.abbott/en-ph/support/free-trial.html');
});

/**
```

```
 * Fill text inputs.
 */
When('I type {string} into the First name field', async (firstName: string) => {
  await page.fill('input[name="firstName"]', firstName);
});

When('I type {string} into the Last name field', async (lastName: string) => {
  await page.fill('input[name="lastName"]', lastName);
});

When('I type {string} into the Email field', async (email: string) => {
  await page.fill('input[name="emailAddress"]', email);
});

When('I type {string} into the Mobile number field', async (mobile: string) => {
  // The field already contains the default country code (+63). Replace it.
  const mobileInput = page.locator('input[name="mobileNumber"]');
  await mobileInput.fill(''); // clear existing value
  await mobileInput.type(mobile);
});

/**
 * Dropdown selections.
 */
When(
  'I select {string} from the {string} dropdown',
  async (option: string, dropdownLabel: string) => {
    await selectFromDropdown(dropdownLabel, option);
```

```
  }
);

/**
 * Consent checkbox.
 */
When('I check the consent checkbox', async () => {
  const checkbox = page.locator('input[name="consentToContact"]');
  await expect(checkbox).toBeVisible();
  const isChecked = await checkbox.isChecked();
  if (!isChecked) {
    await checkbox.check();
  }
  await expect(checkbox).toBeChecked();
});

/**
 * Submit the form.
 */
When('I click the Submit button', async () => {
  // Adjust the selector if the actual button text differs.
  const submitBtn = page.locator('button:has-text("Submit")');
  await submitBtn.waitFor({ state: 'visible', timeout: 5000 });
  await submitBtn.click();
});

/**
 * Verify confirmation message.
```

```
 */
Then(
  'I should see a confirmation message containing {string}',
  async (expectedText: string) => {
    // Wait for a generic success container – adjust selector as needed.
    const confirmation = page.locator(` text=${expectedText}` );
    await confirmation.waitFor({ state: 'visible', timeout: 10000 });
    await expect(confirmation).toBeVisible();

    // Clean-up
    await browser.close();
  }
);
```